

reglas_asociacion

April 12, 2026

1 Minería de reglas de asociación

```
[43]: from pathlib import Path
import csv
import random

import pandas as pd
import matplotlib.pyplot as plt

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules, fpmax
```

1.1 Antes de empezar: ideas clave

En minería de reglas de asociación se trabaja con transacciones. Cada transacción contiene un conjunto de artículos comprados juntos.

Ejemplo:

- T1 = {pan, leche}
- T2 = {pan, pañales, cerveza, huevos}
- T3 = {leche, pañales, cerveza, refresco}

A partir de esas transacciones buscamos patrones del tipo:

{A} -> {B} : cuando aparece A, suele aparecer también B

Las tres medidas más importantes son:

1.2 1. Soporte

Mide qué proporción de transacciones contiene el conjunto de ítems (A).

$$support(A) = \frac{\text{número de transacciones que contienen } A}{\text{número total de transacciones}}$$

1.3 2. Confianza

Mide qué tan frecuentemente aparece el consecuente cuando ya apareció el antecedente.

$$confidence(A \rightarrow B) = \frac{support(A \cup B)}{support(A)}$$

1.4 3. Lift

Mide si la relación entre antecedente y consecuente es mayor a la esperada por azar.

$$lift(A \rightarrow B) = \frac{confidence(A \rightarrow B)}{support(B)}$$

Interpretación básica del lift:

- **lift** > 1: asociación positiva
- **lift** = 1: independencia aproximada
- **lift** < 1: asociación negativa o poco interesante

1.5 Ejemplo 01

Se tienen las siguientes transacciones:

T1 = {pan, leche}
T2 = {pan, pañales, cerveza, huevos}
T3 = {leche, pañales, cerveza, refresco de cola}
T4 = {pan, leche, pañales, cerveza}
T5 = {pan, leche, pañales, refresco de cola}

Con este ejemplo calcularemos manualmente la regla:

{pañales} -> {cerveza}

```
[47]: transacciones = [
    ["pan", "leche"],
    ["pan", "pañales", "cerveza", "huevos"],
    ["leche", "pañales", "cerveza", "refresco"],
    ["pan", "leche", "pañales", "cerveza"],
    ["pan", "leche", "pañales", "refresco"],
]

codificador = TransactionEncoder()
arreglo = codificador.fit(transacciones).transform(transacciones)
matriz_binaria = pd.DataFrame(arreglo, columns=codificador.columns_)

print("Matriz:")
matriz_binaria
```

Matriz:

```
[47]:   cerveza  huevos  leche   pan  pañales  refresco
0    False   False   True   True    False    False
1     True    True   False   True     True    False
```

2	True	False	True	False	True	True
3	True	False	True	True	True	False
4	False	False	True	True	True	True

```
[48]: soporte_items = matriz_binaria.mean(axis=0).sort_values(ascending=False).
      ↪rename("soporte") #
      conteo_items = matriz_binaria.sum(axis=0).sort_values(ascending=False).
      ↪rename("cuantos")

      resumen = pd.concat([conteo_items, soporte_items], axis=1)
      print("Soporte de artículos individuales:")
      resumen
```

Soporte de artículos individuales:

```
[48]:
```

	cuantos	soporte
pan	4	0.8
leche	4	0.8
pañales	4	0.8
cerveza	3	0.6
refresco	2	0.4
huevos	1	0.2

1.5.1 Cálculo la regla {pañales} -> {cerveza}

- pañales aparece en T2, T3, T4 y T5 -> 4 transacciones
- cerveza aparece en T2, T3 y T4 -> 3 transacciones
- {pañales, cerveza} aparece en T2, T3 y T4 -> 3 transacciones

Como tenemos 5 transacciones en total:

- $\text{soporte}(\text{pañales}) = 4 / 5 = 0.80$
- $\text{soporte}(\text{cerveza}) = 3 / 5 = 0.60$
- $\text{soporte}(\{\text{pañales}, \text{cerveza}\}) = 3 / 5 = 0.60$

Entonces:

- $\text{confianza}(\{\text{pañales}\} \rightarrow \{\text{cerveza}\}) = 0.60 / 0.80 = 0.75$
- $\text{lift}(\{\text{pañales}\} \rightarrow \{\text{cerveza}\}) = 0.75 / 0.60 = 1.25$

Interpretación:

- La confianza es 0.75, así que el 75% de las veces que aparece **pañales**, también aparece **cerveza**.
- El lift es 1.25, así que la relación es más fuerte que una coincidencia al azar.

```
[49]: total_transacciones = len(matriz_binaria)

      soporte_pañales = matriz_binaria["pañales"].mean()
      soporte_cerveza = matriz_binaria["cerveza"].mean()
```

```

soporte_pañales_cerveza = matriz_binaria[["pañales", "cerveza"]].all(axis=1).
↳mean()

confianza_pañales_a_cerveza = soporte_pañales_cerveza / soporte_pañales
lift_pañales_a_cerveza = confianza_pañales_a_cerveza / soporte_cerveza

resumen_regla_manual = pd.DataFrame({
    "metrica": [
        "soporte(pañales)",
        "soporte(cerveza)",
        "soporte(pañales & cerveza)",
        "confianza(pañales -> cerveza)",
        "lift(pañales -> cerveza)"
    ],
    "valor": [
        soporte_pañales,
        soporte_cerveza,
        soporte_pañales_cerveza,
        confianza_pañales_a_cerveza,
        lift_pañales_a_cerveza
    ]
})

resumen_regla_manual

```

```

[49]:

```

	metrica	valor
0	soporte(pañales)	0.80
1	soporte(cerveza)	0.60
2	soporte(pañales & cerveza)	0.60
3	confianza(pañales -> cerveza)	0.75
4	lift(pañales -> cerveza)	1.25

Reglas de asociacion para el dataset Groceries.

1.6 1. Leer el archivo Groceries

```

[51]: ruta_archivo = Path("./data/raw/groceries_transactions.csv")

transacciones = []

with open(ruta_archivo, "r", encoding="utf-8") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        articulos = [articulo.strip().strip('"') for articulo in fila if
↳articulo.strip()]
        articulos = list(dict.fromkeys(articulos)) # Eliminar duplicados
↳dentro de la transacción

```

```

        if articulos:
            transacciones.append(articulos)

print("Número de transacciones:" , len(transacciones))
print("Primera transacción:" , transacciones[0])

```

Número de transacciones: 9835

Primera transacción: ['citrus fruit', 'semi-finished bread', 'margarine', 'ready soups']

1.7 2. Conversión a matriz binaria

- True o 1 si el artículo aparece en la transacción
- False o 0 si no aparece

```

[58]: codificador = TransactionEncoder()
arreglo_codificado = codificador.fit(transacciones).transform(transacciones)
datos_binarios = pd.DataFrame(arreglo_codificado, columns=codificador.columns_)
datos_binarios

```

```

[58]:
Instant food products  UHT-milk  abrasive cleaner  artif. sweetener \
0                      False      False              False      False
1                      False      False              False      False
2                      False      False              False      False
3                      False      False              False      False
4                      False      False              False      False
...
9830                   False      False              False      False
9831                   False      False              False      False
9832                   False      False              False      False
9833                   False      False              False      False
9834                   False      False              False      False

baby cosmetics  baby food  bags  baking powder  bathroom cleaner \
0              False      False  False          False          False
1              False      False  False          False          False
2              False      False  False          False          False
3              False      False  False          False          False
4              False      False  False          False          False
...
9830            False      False  False          False          False
9831            False      False  False          False          False
9832            False      False  False          False          False
9833            False      False  False          False          False
9834            False      False  False          False          False

beef  ...  turkey  vinegar  waffles  whipped/sour cream  whisky \
0    False  ...   False   False   False              False   False

```

1	False	...	False	False	False	False	False
2	False	...	False	False	False	False	False
3	False	...	False	False	False	False	False
4	False	...	False	False	False	False	False
...	...	...	...	...	...	...	...
9830	True	...	False	False	False	True	False
9831	False	...	False	False	False	False	False
9832	False	...	False	False	False	False	False
9833	False	...	False	False	False	False	False
9834	False	...	False	True	False	False	False

	white bread	white wine	whole milk	yogurt	zwieback
0	False	False	False	False	False
1	False	False	False	True	False
2	False	False	True	False	False
3	False	False	False	True	False
4	False	False	True	False	False
...	...	...	...	...	...
9830	False	False	True	False	False
9831	False	False	False	False	False
9832	False	False	False	True	False
9833	False	False	False	False	False
9834	False	False	False	False	False

[9835 rows x 169 columns]

```
[ ]: numero_transacciones = len(datos_binarios)
numero_articulos = datos_binarios.shape[1]
proporcion_de_unos = datos_binarios.to_numpy().mean()

print("Transacciones:", numero_transacciones)
print("Artículos distintos:", numero_articulos)
print("Porcentaje de presencia:", round(proporcion_de_unos, 6))
```

Transacciones: 9835
 Artículos distintos: 169
 Porcentaje de presencia: 0.026091

1.8 3. Exploración del dataset

```
[53]: frecuencia_absoluta = datos_binarios.sum(axis=0).sort_values(ascending=False)
frecuencia_relativa = datos_binarios.mean(axis=0).sort_values(ascending=False)

resumen_articulos = pd.DataFrame({
    "frecuencia_absoluta": frecuencia_absoluta,
    "frecuencia_relativa": frecuencia_relativa
})
```

```
print("Los 10 artículos más frecuentes son:")
resumen_articulos.head(10)
```

Los 10 artículos más frecuentes son:

```
[53]:
```

	frecuencia_absoluta	frecuencia_relativa
whole milk	2513	0.255516
other vegetables	1903	0.193493
rolls/buns	1809	0.183935
soda	1715	0.174377
yogurt	1372	0.139502
bottled water	1087	0.110524
root vegetables	1072	0.108998
tropical fruit	1032	0.104931
shopping bags	969	0.098526
sausage	924	0.093950

1.9 4. Exploración de las primeras 5 transacciones

```
[9]: for indice, transaccion in enumerate(transacciones[:5], start=1):
      print(f"[{indice}] {transaccion}")
```

```
[1] ['citrus fruit', 'semi-finished bread', 'margarine', 'ready soups']
[2] ['tropical fruit', 'yogurt', 'coffee']
[3] ['whole milk']
[4] ['pip fruit', 'yogurt', 'cream cheese', 'meat spreads']
[5] ['other vegetables', 'whole milk', 'condensed milk', 'long life bakery
product']
```

1.10 5. Visualización de transacciones como matriz dispersa

```
[55]: def graficar_transacciones_dispersas(datos: pd.DataFrame, numero_filas: int = 15,
      titulo: str = ""):
      muestra = datos.iloc[:numero_filas]
      coordenadas_y, coordenadas_x = muestra.to_numpy().nonzero()

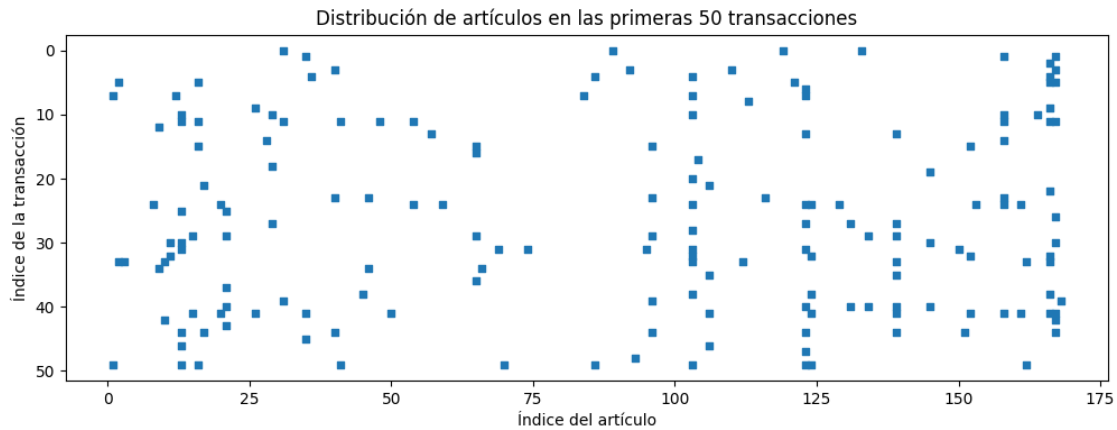
      plt.figure(figsize=(12, 4))
      plt.scatter(coordenadas_x, coordenadas_y, marker="s", s=20)
      plt.gca().invert_yaxis()
      plt.xlabel("Índice del artículo")
      plt.ylabel("Índice de la transacción")
      plt.title(titulo or f"Distribución de artículos en las primeras {numero_filas} transacciones")
      plt.show()
```

```
[62]: graficar_transacciones_dispersas(
      datos_binarios,
```

```

numero_filas=50,
titulo="Distribución de artículos en las primeras 50 transacciones"
)

```



```

[63]: def graficar_transacciones_con_nombres(datos: pd.DataFrame, numero_filas: int = 5,
      ↪ titulo: str = ""):
    muestra = datos.iloc[:numero_filas]

    articulos_presentes = muestra.columns[muestra.any(axis=0)]
    muestra_filtrada = muestra[articulos_presentes]

    coordenadas_y, coordenadas_x = muestra_filtrada.to_numpy().nonzero()

    plt.figure(figsize=(12, 4))
    plt.scatter(coordenadas_x, coordenadas_y, marker="s", s=80)
    plt.gca().invert_yaxis()

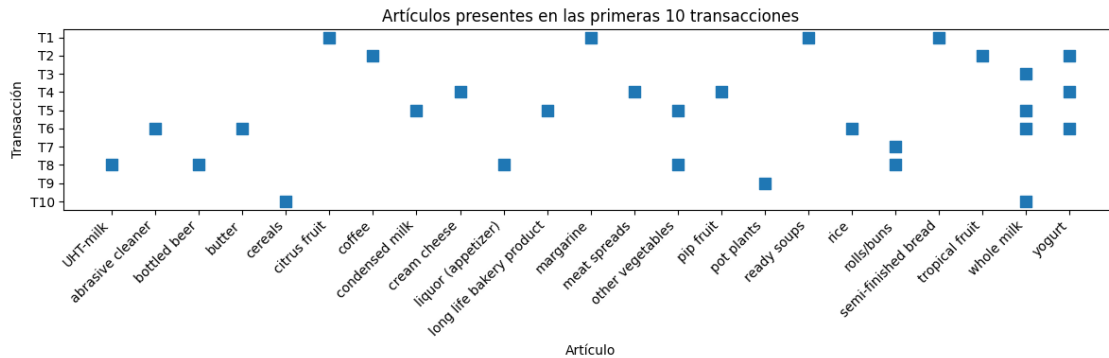
    plt.xticks(
        ticks=range(len(articulos_presentes)),
        labels=articulos_presentes,
        rotation=45,
        ha="right"
    )

    plt.yticks(range(numero_filas), [f"T{i+1}" for i in range(numero_filas)])
    plt.xlabel("Artículo")
    plt.ylabel("Transacción")
    plt.title(titulo or f"Artículos presentes en las primeras {numero_filas} ↪
      ↪ transacciones")
    plt.tight_layout()
    plt.show()

```



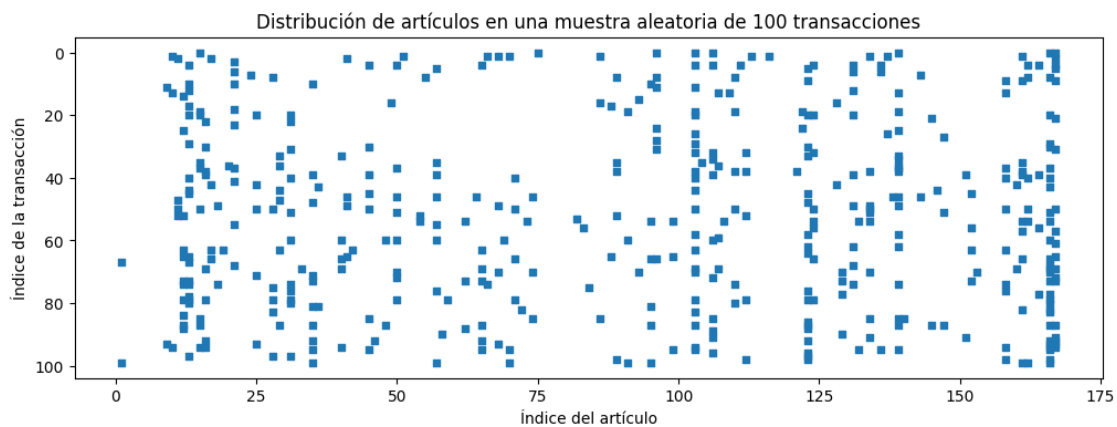
```
[66]: graficar_transacciones_con_nombres(
      datos=datos_binarios,
      numero_filas=10,
      titulo="Artículos presentes en las primeras 10 transacciones"
    )
```



1.11 6. Muestra aleatoria de 100 transacciones

```
[81]: tamano_muestra = min(100, len(datos_binarios))
      indices_muestra = random.sample(range(len(datos_binarios)), tamano_muestra)
      datos_muestra = datos_binarios.iloc[indices_muestra].reset_index(drop=True)

      graficar_transacciones_dispersas(
          datos_muestra,
          numero_filas=tamano_muestra,
          titulo="Distribución de artículos en una muestra aleatoria de 100_
      ↪transacciones"
      )
```

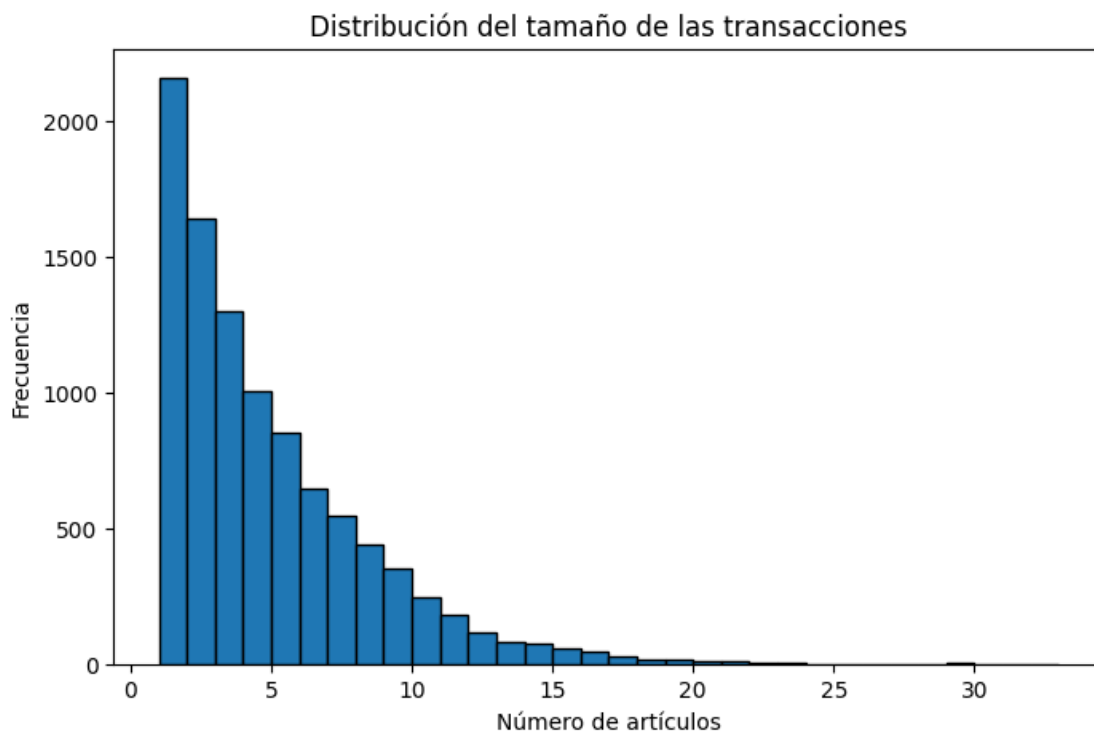


1.12 7. Tamaño de las transacciones

```
[ ]: tamaño_transacciones = datos_binarios.sum(axis=1) #axis 1 = suma por fila ,  
    ↪axis = 0 ; suma pr columnas  
tamaño_transacciones.describe()
```

```
[ ]: count    9835.000000  
     mean      4.409456  
     std       3.589385  
     min       1.000000  
     25%       2.000000  
     50%       3.000000  
     75%       6.000000  
     max      32.000000  
     dtype: float64
```

```
[85]: plt.figure(figsize=(8, 5))  
      plt.hist(tamaño_transacciones, bins=range(1, int(tamaño_transacciones.max()) +  
    ↪2), edgecolor="black")  
      plt.title("Distribución del tamaño de las transacciones")  
      plt.xlabel("Número de artículos")  
      plt.ylabel("Frecuencia")  
      plt.show()
```



```
[89]: indice = tamano_transacciones[tamano_transacciones == 32].index[0]

fila = datos_binarios.loc[indice]
articulos = fila[fila].index.tolist()

print("Cantidad de artículos:", len(articulos))
print(articulos)
```

Cantidad de artículos: 32

```
['beef', 'beverages', 'butter', 'candles', 'chicken', 'citrus fruit', 'cream
cheese', 'curd', 'domestic eggs', 'flour', 'frankfurter', 'ham', 'hard cheese',
'hygiene articles', 'liver loaf', 'margarine', 'mayonnaise', 'other vegetables',
'pasta', 'roll products', 'rolls/buns', 'root vegetables', 'sausage', 'skin
care', 'soft cheese', 'soups', 'specialty fat', 'sugar', 'tropical fruit',
'whipped/sour cream', 'whole milk', 'yogurt']
```

1.13 8. Frecuencia de artículos

```
[90]: print("Frecuencia relativa de los primeros 4 artículos:")
print(datos_binarios.iloc[:, :4].mean(axis=0))

print("\nTop 10 artículos por frecuencia absoluta:")
print(frecuencia_absoluta.head(10))

print("\nArtículos menos frecuentes:")
print(frecuencia_absoluta.tail(10))
```

Frecuencia relativa de los primeros 4 artículos:

Instant food products	0.008033
UHT-milk	0.033452
abrasive cleaner	0.003559
artif. sweetener	0.003254
dtype: float64	

Top 10 artículos por frecuencia absoluta:

whole milk	2513
other vegetables	1903
rolls/buns	1809
soda	1715
yogurt	1372
bottled water	1087
root vegetables	1072
tropical fruit	1032
shopping bags	969
sausage	924
dtype: int64	

Artículos menos frecuentes:

```

make up remover      8
salad dressing       8
toilet cleaner       7
frozen chicken       6
baby cosmetics       6
bags                 4
kitchen utensil      4
preservation products 2
baby food            1
sound storage medium 1
dtype: int64

```

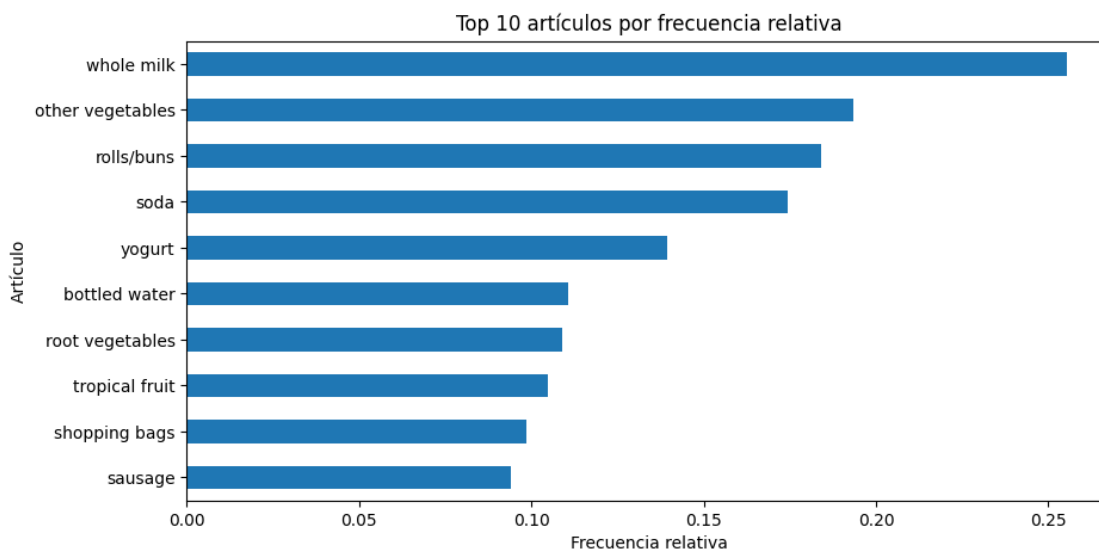
1.14 9. Gráficas de frecuencia

```

[93]: top_n = 10

plt.figure(figsize=(10, 5))
frecuencia_relativa.head(top_n).sort_values().plot(kind="barh")
plt.title("Top 10 artículos por frecuencia relativa")
plt.xlabel("Frecuencia relativa")
plt.ylabel("Artículo")
plt.show()

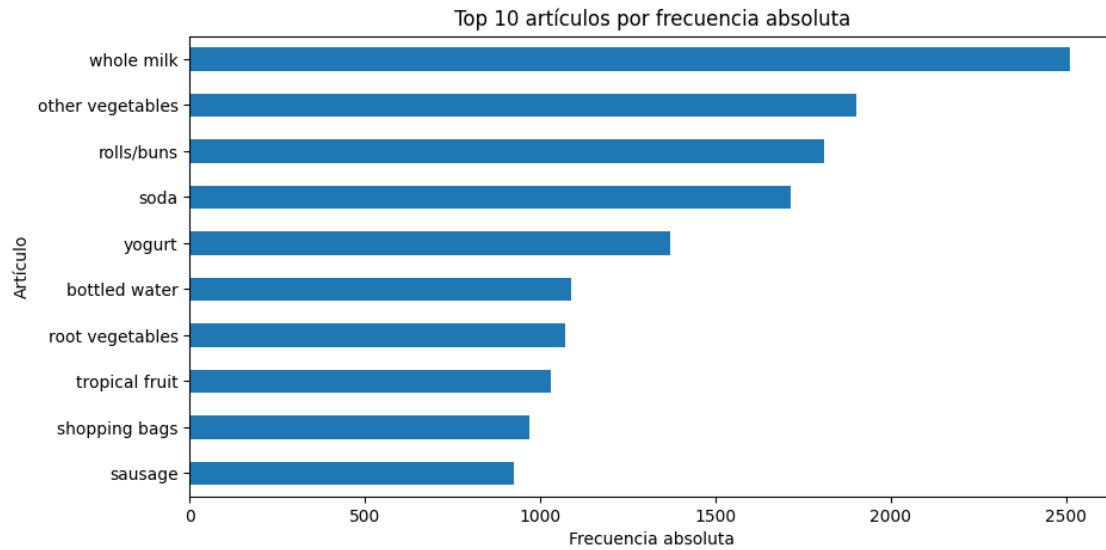
```



```

[94]: plt.figure(figsize=(10, 5))
frecuencia_absoluta.head(top_n).sort_values().plot(kind="barh")
plt.title("Top 10 artículos por frecuencia absoluta")
plt.xlabel("Frecuencia absoluta")
plt.ylabel("Artículo")
plt.show()

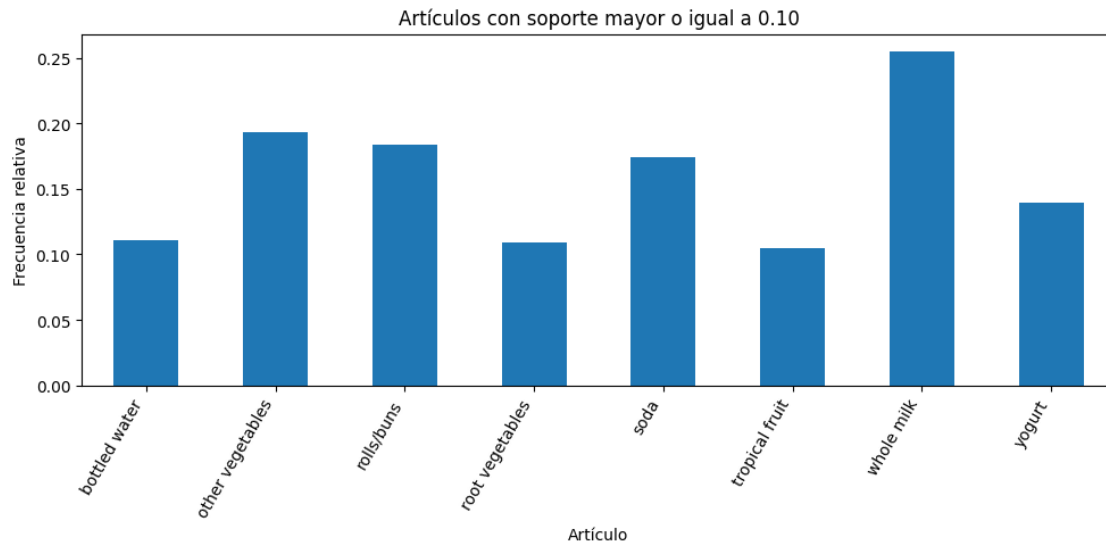
```



1.15 10. Artículos con soporte mayor o igual a 0.10

```
[97]: umbral_soporte = 0.1
articulos_soporte_10 = frecuencia_relativa[frecuencia_relativa >=
↳ umbral_soporte].sort_index()

plt.figure(figsize=(10, 5))
articulos_soporte_10.plot(kind="bar")
plt.title("Artículos con soporte mayor o igual a 0.10")
plt.xlabel("Artículo")
plt.ylabel("Frecuencia relativa")
plt.xticks(rotation=60, ha="right")
plt.tight_layout()
plt.show()
```



1.16 11. Itemsets frecuentes con soporte mínimo de 0.10

Se buscan los conjuntos de artículos que aparecen en al menos el 10% de las transacciones.

- `support = 0.10` significa **10% del total de transacciones**
- `conteo` es el número aproximado de transacciones donde aparece el itemset
- `longitud` indica cuántos artículos contiene el itemset

```
[19]: itemsets_frecuentes_10 = apriori(datos_binarios, min_support=0.10,
    ↪ use_colnames=True)
itemsets_frecuentes_10["conteo"] = (itemsets_frecuentes_10["support"] *
    ↪ len(datos_binarios)).round().astype(int)
itemsets_frecuentes_10["longitud"] = itemsets_frecuentes_10["itemsets"].
    ↪ apply(len)

itemsets_frecuentes_10.sort_values(["longitud", "support"], ascending=[True,
    ↪ False]).head(20)
```

```
[19]:
```

	support	itemsets	conteo	longitud
6	0.255516	frozenset({whole milk})	2513	1
1	0.193493	frozenset({other vegetables})	1903	1
2	0.183935	frozenset({rolls/buns})	1809	1
4	0.174377	frozenset({soda})	1715	1
7	0.139502	frozenset({yogurt})	1372	1
0	0.110524	frozenset({bottled water})	1087	1
3	0.108998	frozenset({root vegetables})	1072	1
5	0.104931	frozenset({tropical fruit})	1032	1

1.17 12. Itemsets frecuentes con soporte mínimo de 0.05 y longitud mayor o igual a 2

```
[98]: itemsets_frecuentes_05 = apriori(datos_binarios, min_support=0.05,
    ↪ use_colnames=True)
itemsets_frecuentes_05["longitud"] = itemsets_frecuentes_05["itemsets"].
    ↪ apply(len)
itemsets_frecuentes_05["conteo"] = (itemsets_frecuentes_05["support"] *
    ↪ len(datos_binarios)).round().astype(int)

itemsets_frecuentes_05_filtrados =
    ↪ itemsets_frecuentes_05[itemsets_frecuentes_05["longitud"] >= 2]
itemsets_frecuentes_05_filtrados.sort_values("support", ascending=False).
    ↪ head(20)
```

```
[98]:
```

	support	itemsets	longitud	conteo
28	0.074835	frozenset({whole milk, other vegetables})	2	736
29	0.056634	frozenset({rolls/buns, whole milk})	2	557
30	0.056024	frozenset({yogurt, whole milk})	2	551

1.18 13. Limpieza de datos: eliminar transacciones con un solo artículo

```
[99]: mascara_mas_de_un_articulo = tamaño_transacciones > 1
transacciones_limpias = [
    transacciones[i]
    for i in range(len(transacciones))
    if mascara_mas_de_un_articulo.iloc[i]
]

codificador_limpio = TransactionEncoder()
arreglo_limpio = codificador_limpio.fit(transacciones_limpias).
    ↪ transform(transacciones_limpias)
datos_limpios = pd.DataFrame(arreglo_limpio, columns=codificador_limpio.
    ↪ columns_)

print(f"Transacciones originales: {len(datos_binarios)}")
print(f"Transacciones con más de un artículo: {len(datos_limpios)}")
```

Transacciones originales: 9835

Transacciones con más de un artículo: 7676

1.19 14. Primer intento de reglas: soporte = 0.10 y confianza = 0.80

Este primer intento usa umbrales exigentes:

- soporte mínimo = 0.10
- confianza mínima = 0.80

En datasets reales esto puede producir muy pocas reglas o incluso ninguna, porque estamos pidiendo patrones muy frecuentes y además muy confiables.

```
[100]: itemsets_modelo_inicial = apriori(datos_limpios, min_support=0.10,
    ↪use_colnames=True)
reglas_iniciales = association_rules(itemsets_modelo_inicial,
    ↪metric="confidence", min_threshold=0.80)

if not reglas_iniciales.empty:
    reglas_iniciales["longitud_regla"] = (
        reglas_iniciales["antecedents"].apply(len) +
    ↪reglas_iniciales["consequents"].apply(len)
    )
    reglas_iniciales = reglas_iniciales[reglas_iniciales["longitud_regla"] >=
    ↪2].copy()

print(f"Número de reglas con soporte >= 0.10 y confianza >= 0.80:
    ↪{len(reglas_iniciales)}")

if reglas_iniciales.empty:
    print("No se generaron reglas con estos umbrales. Esto no es un error:
    ↪significa que los criterios fueron muy estrictos.")
```

Número de reglas con soporte >= 0.10 y confianza >= 0.80: 0

No se generaron reglas con estos umbrales. Esto no es un error: significa que los criterios fueron muy estrictos.

1.20 15. Modelo ajustado: soporte = 0.008 y confianza = 0.30

- soporte mínimo = 0.008
- confianza mínima = 0.30

Esto permite encontrar asociaciones menos frecuentes, pero todavía potencialmente útiles.

Nota importante: bajar mucho el soporte puede generar demasiadas reglas.

Por eso siempre conviene revisar después métricas como lift y no quedarse solo con la confianza.

```
[101]: itemsets_modelo = apriori(datos_limpios, min_support=0.008, use_colnames=True)
reglas = association_rules(itemsets_modelo, metric="confidence",
    ↪min_threshold=0.30)

if not reglas.empty:
    reglas["longitud_regla"] = (
        reglas["antecedents"].apply(len) + reglas["consequents"].apply(len)
    )
    reglas = reglas[reglas["longitud_regla"] >= 2].copy()
    reglas["conteo"] = (reglas["support"] * len(datos_limpios)).round().
    ↪astype(int)
```



```
print(f"Número de reglas generadas: {len(reglas)}")
```

Número de reglas generadas: 322

1.21 16. Función auxiliar para mostrar reglas de manera más clara

La función siguiente convierte los conjuntos `frozenset(...)` en texto legible y selecciona las columnas más importantes para interpretar cada regla.

```
[24]: def table_rules(rules_df: pd.DataFrame) -> pd.DataFrame:
        output = rules_df.copy()
        output["antecedent"] = output["antecedents"].apply(lambda x: ", ".
        ↪join(sorted(list(x))))
        output["consequent"] = output["consequents"].apply(lambda x: ", ".
        ↪join(sorted(list(x))))
        selected_columns = [
            "antecedent",
            "consequent",
            "antecedent support",
            "consequent support",
            "support",
            "confidence",
            "lift",
            "conteo",
        ]
        return output[selected_columns].sort_values(
            ["lift", "confidence", "support"], ascending=[False, False, False]
        )
```

1.22 17. Primeras 10 reglas

```
[25]: table_rules(reglas).head(10)
```

```
[25]:
```

	antecedent \			
312	root vegetables, tropical fruit			
53	herbs			
317	root vegetables, yogurt			
309	other vegetables, tropical fruit, whole milk			
107	beef, other vegetables			
112	beef, whole milk			
216	other vegetables, pip fruit			
272	pip fruit, yogurt			
147	citrus fruit, other vegetables			
320	other vegetables, whole milk, yogurt			

	consequent	antecedent support	consequent support \
312	other vegetables, whole milk	0.026967	0.095883
53	root vegetables	0.020453	0.136399

317	other vegetables, whole milk	0.033090	0.095883
309	root vegetables	0.021886	0.136399
107	root vegetables	0.025274	0.136399
112	root vegetables	0.027228	0.136399
216	tropical fruit	0.033481	0.131449
272	tropical fruit	0.023059	0.131449
147	root vegetables	0.036998	0.136399
320	tropical fruit	0.028530	0.131449

	support	confidence	lift	conteo
312	0.008989	0.333333	3.476449	69
53	0.008989	0.439490	3.222090	69
317	0.010031	0.303150	3.161653	77
309	0.008989	0.410714	3.011120	69
107	0.010162	0.402062	2.947686	78
112	0.010292	0.377990	2.771208	79
216	0.012116	0.361868	2.752920	93
272	0.008207	0.355932	2.707766	63
147	0.013288	0.359155	2.633117	102
320	0.009771	0.342466	2.605319	75

1.23 Nota

antecedent -> consequent

- **antecedent support**: proporción de transacciones que contienen el antecedente
- **consequent support**: proporción de transacciones que contienen el consecuente
- **support**: proporción de transacciones que contienen ambos
- **confidence**: de las transacciones que tienen el antecedente, cuántas también tienen el consecuente
- **lift**: qué tanto mejora esa coocurrencia frente a lo esperado por azar

```
[26]: def explain_rule(rule_row: pd.Series, binary_data: pd.DataFrame) -> pd.
      DataFrame:
      antecedent = sorted(list(rule_row["antecedents"]))
      consequent = sorted(list(rule_row["consequents"]))
      union_items = antecedent + [item for item in consequent if item not in_
      antecedent]

      antecedent_mask = binary_data[antecedent].all(axis=1)
      consequent_mask = binary_data[consequent].all(axis=1)
      union_mask = binary_data[union_items].all(axis=1)

      total_rows = len(binary_data)
      antecedent_count = int(antecedent_mask.sum())
      consequent_count = int(consequent_mask.sum())
      union_count = int(union_mask.sum())
```

```

    antecedent_support = antecedent_count / total_rows
    consequent_support = consequent_count / total_rows
    union_support = union_count / total_rows
    confidence = union_support / antecedent_support if antecedent_support > 0
    ↪else float("nan")
    lift = confidence / consequent_support if consequent_support > 0 else
    ↪float("nan")

    explanation = pd.DataFrame({
        "concepto": [
            "Antecedente",
            "Consecuente",
            "Transacciones con antecedente",
            "Transacciones con consecuente",
            "Transacciones con ambos",
            "Support del antecedente",
            "Support del consecuente",
            "Support de la regla",
            "Confidence",
            "Lift",
        ],
        "valor": [
            ", ".join(antecedent),
            ", ".join(consequent),
            antecedent_count,
            consequent_count,
            union_count,
            round(antecedent_support, 6),
            round(consequent_support, 6),
            round(union_support, 6),
            round(confidence, 6),
            round(lift, 6),
        ]
    })
    return explanation

regla_ejemplo = reglas.sort_values(["lift", "confidence", "support"],
    ↪ascending=[False, False, False]).iloc[0]
explain_rule(regla_ejemplo, datos_limpios)

```

[26]:

	concepto	valor
0	Antecedente	root vegetables, tropical fruit
1	Consecuente	other vegetables, whole milk
2	Transacciones con antecedente	207
3	Transacciones con consecuente	736
4	Transacciones con ambos	69
5	Support del antecedente	0.026967

6	Support del consecuente	0.095883
7	Support de la regla	0.008989
8	Confidence	0.333333
9	Lift	3.476449

La tabla anterior permite comprobar que las métricas de la regla sí salen directamente de los conteos del dataset.

Ese paso es importante didácticamente porque evita la idea de que Apriori “inventa” reglas; en realidad, solo automatiza conteos y filtros.

1.24 19. Resumen estadístico de las reglas

```
[27]: print("Distribución de la longitud de las reglas:")
      print(reglas["longitud_regla"].value_counts().sort_index())

      print("\nResumen de medidas de calidad:")
      reglas[["support", "confidence", "lift", "conteo"]].describe()
```

Distribución de la longitud de las reglas:

longitud_regla

2 106

3 202

4 14

Name: count, dtype: int64

Resumen de medidas de calidad:

```
[27]:
```

	support	confidence	lift	conteo
count	322.000000	322.000000	322.000000	322.000000
mean	0.015795	0.416646	1.698731	121.242236
std	0.011877	0.083455	0.407101	91.171420
min	0.008077	0.300000	0.980833	62.000000
25%	0.009771	0.343422	1.402369	75.000000
50%	0.011855	0.410442	1.604515	91.000000
75%	0.016285	0.471798	1.901492	125.000000
max	0.095883	0.660000	3.476449	736.000000

1.25 20. Reglas con mayor lift

```
[28]: reglas_mayor_lift = reglas.sort_values("lift", ascending=False).head(5)
      table_rules(reglas_mayor_lift)
```

```
[28]:
```

	antecedent \
312	root vegetables, tropical fruit
53	herbs
317	root vegetables, yogurt
309	other vegetables, tropical fruit, whole milk
107	beef, other vegetables

	consequent	antecedent	support	consequent support	\
312	other vegetables, whole milk		0.026967	0.095883	
53	root vegetables		0.020453	0.136399	
317	other vegetables, whole milk		0.033090	0.095883	
309	root vegetables		0.021886	0.136399	
107	root vegetables		0.025274	0.136399	

	support	confidence	lift	conteo
312	0.008989	0.333333	3.476449	69
53	0.008989	0.439490	3.222090	69
317	0.010031	0.303150	3.161653	77
309	0.008989	0.410714	3.011120	69
107	0.010162	0.402062	2.947686	78

1.26 21. Reglas con menor lift

Las reglas con lift cercano a 1 suelen ser poco interesantes, porque indican que la presencia del antecedente apenas cambia la probabilidad del consecuente.

Si el lift fuera menor que 1, la relación sería incluso peor que una coincidencia aleatoria.

```
[29]: reglas_menor_lift = reglas.sort_values("lift", ascending=True).head(5)
      table_rules(reglas_menor_lift)
```

```
[29]:
```

	antecedent	consequent	antecedent support	\
248	other vegetables, shopping bags	whole milk	0.029703	
91	rolls/buns	whole milk	0.221470	
94	sausage	whole milk	0.117249	
93	salty snack	whole milk	0.044554	
281	rolls/buns, sausage	whole milk	0.039213	

	consequent support	support	confidence	lift	conteo
248	0.311621	0.009771	0.328947	1.055602	75
91	0.311621	0.072564	0.327647	1.051429	557
94	0.311621	0.038301	0.326667	1.048283	294
93	0.311621	0.014330	0.321637	1.032144	110
281	0.311621	0.011985	0.305648	0.980833	92

1.27 22. Reglas cuyo consecuente sea tropical fruit

```
[30]: reglas_consecuente_tropical = reglas[
      reglas["consequents"].apply(lambda x: x == frozenset({"tropical fruit"}))
    ].sort_values("lift", ascending=False)

      table_rules(reglas_consecuente_tropical).head(20)
```

```
[30]:
```

	antecedent	consequent	\
216	other vegetables, pip fruit	tropical fruit	
272	pip fruit, yogurt	tropical fruit	
320	other vegetables, whole milk, yogurt	tropical fruit	
292	root vegetables, yogurt	tropical fruit	
187	fruit/vegetable juice, other vegetables	tropical fruit	
149	citrus fruit, other vegetables	tropical fruit	
127	bottled water, yogurt	tropical fruit	
311	other vegetables, root vegetables, whole milk	tropical fruit	

	antecedent support	consequent support	support	confidence	lift	\
216	0.033481	0.131449	0.012116	0.361868	2.752920	
272	0.023059	0.131449	0.008207	0.355932	2.707766	
320	0.028530	0.131449	0.009771	0.342466	2.605319	
292	0.033090	0.131449	0.010422	0.314961	2.396073	
187	0.026967	0.131449	0.008468	0.314010	2.388839	
149	0.036998	0.131449	0.011595	0.313380	2.384051	
127	0.029442	0.131449	0.009119	0.309735	2.356315	
311	0.029703	0.131449	0.008989	0.302632	2.302279	

	conteo
216	93
272	63
320	75
292	80
187	65
149	89
127	70
311	69

1.28 23. Reglas cuyo antecedente contenga tropical fruit

```
[31]: reglas_antecedente_tropical = reglas[
    reglas["antecedents"].apply(lambda x: "tropical fruit" in x)
].sort_values("lift", ascending=False)

table_rules(reglas_antecedente_tropical).head(20)
```

```
[31]:
```

	antecedent	\
312	root vegetables, tropical fruit	
309	other vegetables, tropical fruit, whole milk	
321	other vegetables, tropical fruit, whole milk	
237	other vegetables, tropical fruit	
308	root vegetables, tropical fruit, whole milk	
236	root vegetables, tropical fruit	
255	tropical fruit, whipped/sour cream	
293	root vegetables, tropical fruit	

128	bottled water, tropical fruit
318	tropical fruit, whole milk, yogurt
305	tropical fruit, whole milk
286	rolls/buns, tropical fruit
169	curd, tropical fruit
186	fruit/vegetable juice, tropical fruit
319	other vegetables, tropical fruit, yogurt
259	other vegetables, tropical fruit
178	domestic eggs, tropical fruit
215	pip fruit, tropical fruit
148	citrus fruit, tropical fruit
303	tropical fruit, whipped/sour cream

	consequent	antecedent support	consequent support \
312	other vegetables, whole milk	0.026967	0.095883
309	root vegetables	0.021886	0.136399
321	yogurt	0.021886	0.173528
237	root vegetables	0.045987	0.136399
308	other vegetables	0.015373	0.239838
236	other vegetables	0.026967	0.239838
255	other vegetables	0.017718	0.239838
293	yogurt	0.026967	0.173528
128	yogurt	0.023710	0.173528
318	other vegetables	0.019411	0.239838
305	yogurt	0.054195	0.173528
286	yogurt	0.031527	0.173528
169	whole milk	0.013158	0.311621
186	other vegetables	0.017587	0.239838
319	whole milk	0.015763	0.311621
259	yogurt	0.045987	0.173528
178	whole milk	0.014591	0.311621
215	other vegetables	0.026186	0.239838
148	other vegetables	0.025534	0.239838
303	whole milk	0.017718	0.311621

	support	confidence	lift	conteo
312	0.008989	0.333333	3.476449	69
309	0.008989	0.410714	3.011120	69
321	0.009771	0.446429	2.572662	75
237	0.015763	0.342776	2.513037	121
308	0.008989	0.584746	2.438082	69
236	0.015763	0.584541	2.437228	121
255	0.010031	0.566176	2.360658	77
293	0.010422	0.386473	2.227155	80
128	0.009119	0.384615	2.216447	70
318	0.009771	0.503356	2.098728	75
305	0.019411	0.358173	2.064066	149

286	0.011204	0.355372	2.047924	86
169	0.008338	0.633663	2.033445	64
186	0.008468	0.481481	2.007524	65
319	0.009771	0.619835	1.989068	75
259	0.015763	0.342776	1.975338	121
178	0.008859	0.607143	1.948340	68
215	0.012116	0.462687	1.929159	93
148	0.011595	0.454082	1.893281	89
303	0.010162	0.573529	1.840473	78

1.29 24. Reglas que incluyan beef en cualquier lado

```
[32]: reglas_beef = reglas[
    reglas.apply(
        lambda fila: ("beef" in fila["antecedents"]) or ("beef" in
        ↪fila["consequents"]),
        axis=1
    )
].sort_values("lift", ascending=False)

table_rules(reglas_beef).head(20)
```

```
[32]:
```

	antecedent	consequent	antecedent support \
107	beef, other vegetables	root vegetables	0.025274
112	beef, whole milk	root vegetables	0.027228
3	beef	root vegetables	0.063835
106	beef, root vegetables	other vegetables	0.022277
108	beef, whole milk	other vegetables	0.027228
2	beef	other vegetables	0.063835
110	beef, rolls/buns	whole milk	0.017457
109	beef, other vegetables	whole milk	0.025274
113	beef, root vegetables	whole milk	0.022277
111	beef, whole milk	rolls/buns	0.027228
4	beef	whole milk	0.063835

	consequent support	support	confidence	lift	conteo
107	0.136399	0.010162	0.402062	2.947686	78
112	0.136399	0.010292	0.377990	2.771208	79
3	0.136399	0.022277	0.348980	2.558517	171
106	0.239838	0.010162	0.456140	1.901865	78
108	0.239838	0.011855	0.435407	1.815417	91
2	0.239838	0.025274	0.395918	1.650771	194
110	0.311621	0.008729	0.500000	1.604515	67
109	0.311621	0.011855	0.469072	1.505267	91
113	0.311621	0.010292	0.461988	1.482534	79
111	0.221470	0.008729	0.320574	1.447487	67
4	0.311621	0.027228	0.426531	1.368750	209

1.30 25. Reglas que incluyan beef y algún artículo relacionado con vegetable

```
[33]: def contiene_termino_vegetable(conjunto_articulos):
      return any("vegetable" in articulo for articulo in conjunto_articulos)

reglas_beef_vegetables = reglas[
    reglas.apply(
        lambda fila: (
            ("beef" in fila["antecedents"] or "beef" in fila["consequents"])
            and
            (
                contiene_termino_vegetable(fila["antecedents"])
                or contiene_termino_vegetable(fila["consequents"])
            )
        ),
        axis=1
    )
].sort_values("lift", ascending=False)

table_rules(reglas_beef_vegetables).head(20)
```

```
[33]:
```

	antecedent	consequent	antecedent support \
107	beef, other vegetables	root vegetables	0.025274
112	beef, whole milk	root vegetables	0.027228
3	beef	root vegetables	0.063835
106	beef, root vegetables	other vegetables	0.022277
108	beef, whole milk	other vegetables	0.027228
2	beef	other vegetables	0.063835
109	beef, other vegetables	whole milk	0.025274
113	beef, root vegetables	whole milk	0.022277

	consequent support	support	confidence	lift	conteo
107	0.136399	0.010162	0.402062	2.947686	78
112	0.136399	0.010292	0.377990	2.771208	79
3	0.136399	0.022277	0.348980	2.558517	171
106	0.239838	0.010162	0.456140	1.901865	78
108	0.239838	0.011855	0.435407	1.815417	91
2	0.239838	0.025274	0.395918	1.650771	194
109	0.311621	0.011855	0.469072	1.505267	91
113	0.311621	0.010292	0.461988	1.482534	79

1.31 26. Itemsets máximos

En esta parte usamos `fpmmax`, que devuelve **itemsets máximos**.

- un itemset máximo es frecuente y no está contenido en otro itemset frecuente más grande;
- esto **no** significa que sean todos los itemsets frecuentes;
- sirve para resumir patrones grandes sin listar todas sus combinaciones internas.

Por eso esta sección complementa a Apriori, pero no sustituye el análisis completo de reglas.

```
[34]: itemsets_maximos = fpmx(datos_limpios, min_support=0.008, use_colnames=True)
itemsets_maximos["longitud"] = itemsets_maximos["itemsets"].apply(len)
itemsets_maximos["conteo"] = (itemsets_maximos["support"] * len(datos_limpios)).
    ↪round().astype(int)

itemsets_maximos.sort_values(["longitud", "support"], ascending=[False, False]).
    ↪head(20)
```

```
[34]:
```

	support	itemsets	longitud	\
491	0.010031	frozenset({yogurt, whole milk, root vegetables...	4	
474	0.009771	frozenset({yogurt, tropical fruit, whole milk,...	4	
470	0.008989	frozenset({tropical fruit, whole milk, root ve...	4	
500	0.022929	frozenset({rolls/buns, whole milk, other veget...	3	
496	0.019932	frozenset({yogurt, rolls/buns, whole milk})	3	
377	0.018760	frozenset({whipped/sour cream, whole milk, oth...	3	
498	0.017848	frozenset({soda, whole milk, other vegetables})	3	
408	0.017327	frozenset({whole milk, other vegetables, pip f...	3	
433	0.016675	frozenset({citrus fruit, whole milk, other veg...	3	
490	0.016285	frozenset({rolls/buns, whole milk, root vegeta...	3	
335	0.015763	frozenset({domestic eggs, whole milk, other ve...	3	
489	0.015633	frozenset({rolls/buns, root vegetables, other ...	3	
273	0.014721	frozenset({whole milk, butter, other vegetables})	3	
495	0.014721	frozenset({yogurt, rolls/buns, other vegetables})	3	
473	0.014070	frozenset({rolls/buns, tropical fruit, whole m...	3	
376	0.013940	frozenset({yogurt, whipped/sour cream, whole m...	3	
484	0.013809	frozenset({bottled water, whole milk, other ve...	3	
443	0.013549	frozenset({other vegetables, whole milk, pastry})	3	
392	0.013418	frozenset({other vegetables, whole milk, fruit...	3	
494	0.013418	frozenset({yogurt, soda, whole milk})	3	

	conteo
491	77
474	75
470	69
500	176
496	153
377	144
498	137
408	133
433	128
490	125
335	121
489	120
273	113
495	113

473	108
376	107
484	106
443	104
392	103
494	103

1.32 27. Reglas a CSV

```
[104]: reglas_exportar = table_rules(reglas)
reglas_exportar.to_csv("./data/rules/reglas_asociacion_groceries.csv",
    ↪index=False)
```

```
[ ]: rules = pd.read_csv("./data/rules/reglas_asociacion_groceries.csv")

# Validate required columns
required_columns = [
    "antecedent",
    "consequent",
    "antecedent support",
    "consequent support",
    "support",
    "confidence",
    "lift",
    "conteo"
]

missing_columns = [col for col in required_columns if col not in rules.columns]

if missing_columns:
    raise ValueError("Missing columns: " + ", ".join(missing_columns))

min_support = 0.01
min_confidence = 0.45
min_lift = 1.70
min_count = 75

# Filter strong rules
strong_rules = rules[
    (rules["support"] >= min_support) &
    (rules["confidence"] >= min_confidence) &
    (rules["lift"] >= min_lift) &
    (rules["conteo"] >= min_count)
].copy()
```

```

# Sort rules from strongest to weakest
strong_rules = strong_rules.sort_values(
    by=["lift", "confidence", "support", "conteo"],
    ascending=[False, False, False, False]
).reset_index(drop=True)

# Show results
print("Total rules:", len(rules))
print("Strong rules:", len(strong_rules))

strong_rules.head(20)

```

Total rules: 322

Strong rules: 37

```

[ ]:

```

	antecedent	consequent	\
0	citrus fruit, root vegetables	other vegetables	
1	root vegetables, tropical fruit	other vegetables	
2	tropical fruit, whipped/sour cream	other vegetables	
3	root vegetables, whole milk, yogurt	other vegetables	
4	pip fruit, root vegetables	other vegetables	
5	rolls/buns, root vegetables	other vegetables	
6	root vegetables, yogurt	other vegetables	
7	root vegetables, whipped/sour cream	other vegetables	
8	butter, yogurt	whole milk	
9	butter, root vegetables	whole milk	
10	whipped/sour cream, yogurt	other vegetables	
11	chicken, whole milk	other vegetables	
12	root vegetables, whole milk	other vegetables	
13	frozen vegetables, whole milk	other vegetables	
14	onions	other vegetables	
15	other vegetables, root vegetables, yogurt	whole milk	
16	pip fruit, tropical fruit	other vegetables	
17	pork, whole milk	other vegetables	
18	domestic eggs, root vegetables	whole milk	
19	beef, root vegetables	other vegetables	

	antecedent support	consequent support	support	confidence	lift	\
0	0.022668	0.239838	0.013288	0.586207	2.444174	
1	0.026967	0.239838	0.015763	0.584541	2.437228	
2	0.017718	0.239838	0.010031	0.566176	2.360658	
3	0.018629	0.239838	0.010031	0.538462	2.245101	
4	0.019932	0.239838	0.010422	0.522876	2.180117	
5	0.031136	0.239838	0.015633	0.502092	2.093459	

6	0.033090	0.239838	0.016545	0.500000	2.084737
7	0.021886	0.239838	0.010943	0.500000	2.084737
8	0.018760	0.311621	0.011985	0.638889	2.050214
9	0.016545	0.311621	0.010552	0.637795	2.046704
10	0.026576	0.239838	0.013028	0.490196	2.043859
11	0.022538	0.239838	0.010813	0.479769	2.000383
12	0.062663	0.239838	0.029703	0.474012	1.976382
13	0.026186	0.239838	0.012376	0.472637	1.970646
14	0.038953	0.239838	0.018239	0.468227	1.952262
15	0.016545	0.311621	0.010031	0.606299	1.945632
16	0.026186	0.239838	0.012116	0.462687	1.929159
17	0.028400	0.239838	0.013028	0.458716	1.912602
18	0.018369	0.311621	0.010943	0.595745	1.911763
19	0.022277	0.239838	0.010162	0.456140	1.901865

	conteo
0	102
1	121
2	77
3	77
4	80
5	120
6	127
7	84
8	92
9	81
10	100
11	83
12	228
13	95
14	140
15	77
16	93
17	100
18	84
19	78